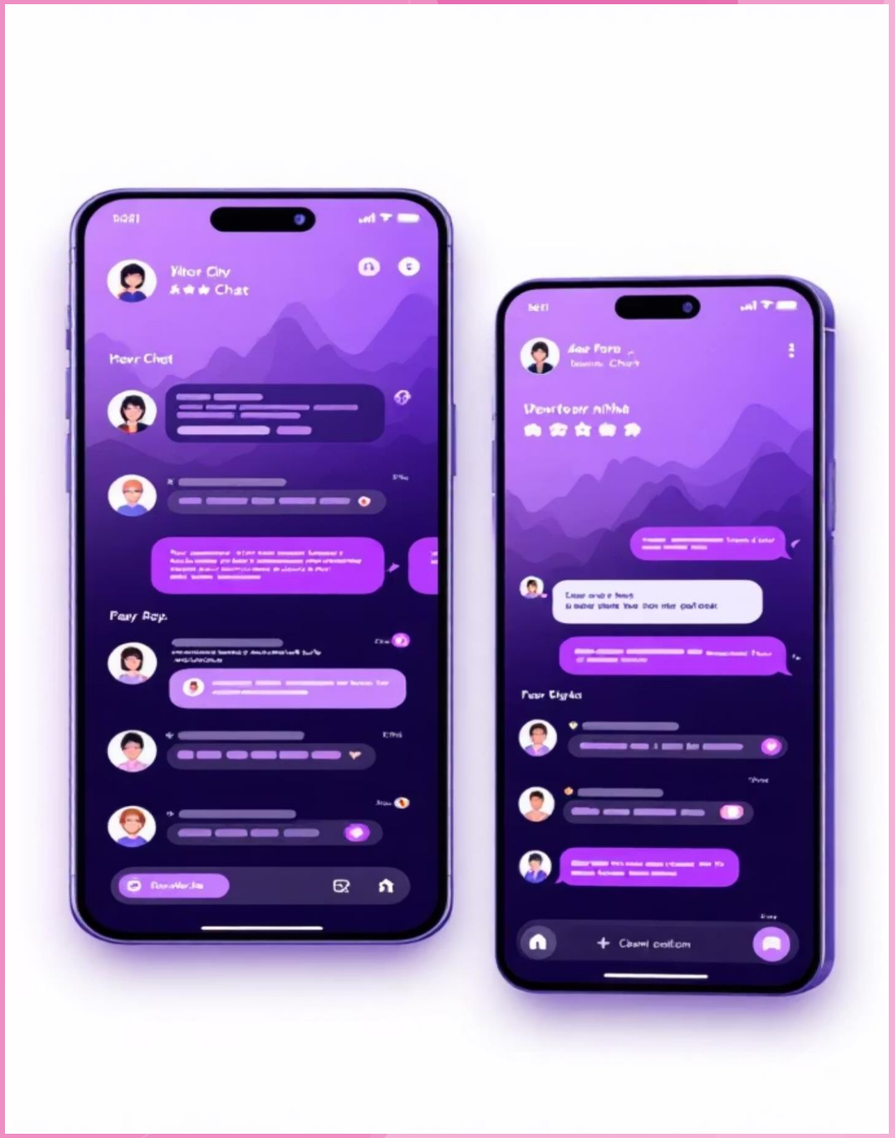
ChatterBox — Real-Time WebSocket Chat Application

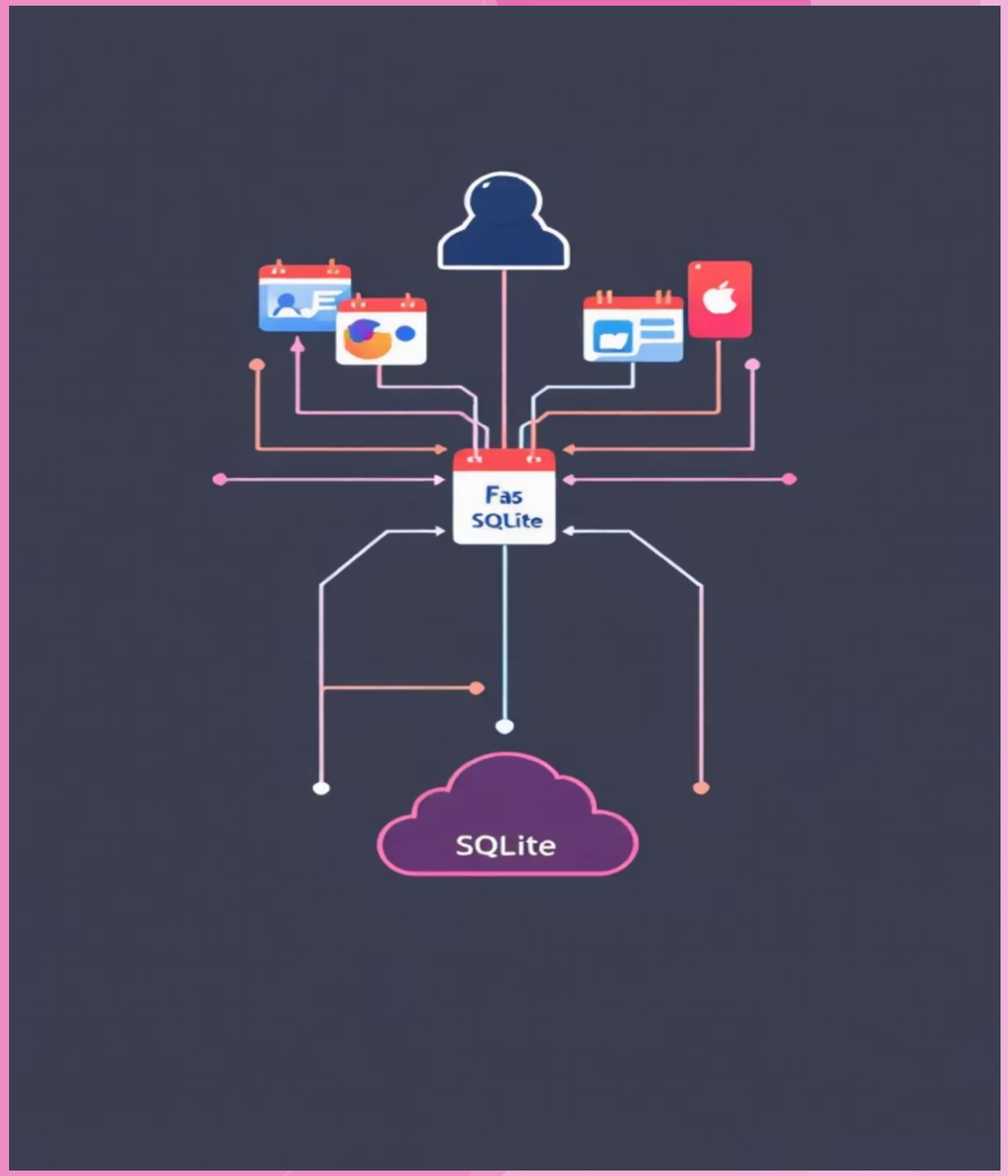
Full-stack demonstration of instant messaging, moderation and persistence.

Presented by: Gopala Bhavya Sri Devi.



Project Overview

ChatterBox is a real-time web chat built with FastAPI and WebSockets. It supports multi-user chat, registration and login, message persistence, seen status, and a toxic-word moderation workflow. The stack demonstrates end-to-end full-stack integration.



Objectives

Real-time Communication

Full duplex messaging with low latency via WebSockets.

Secure Authentication

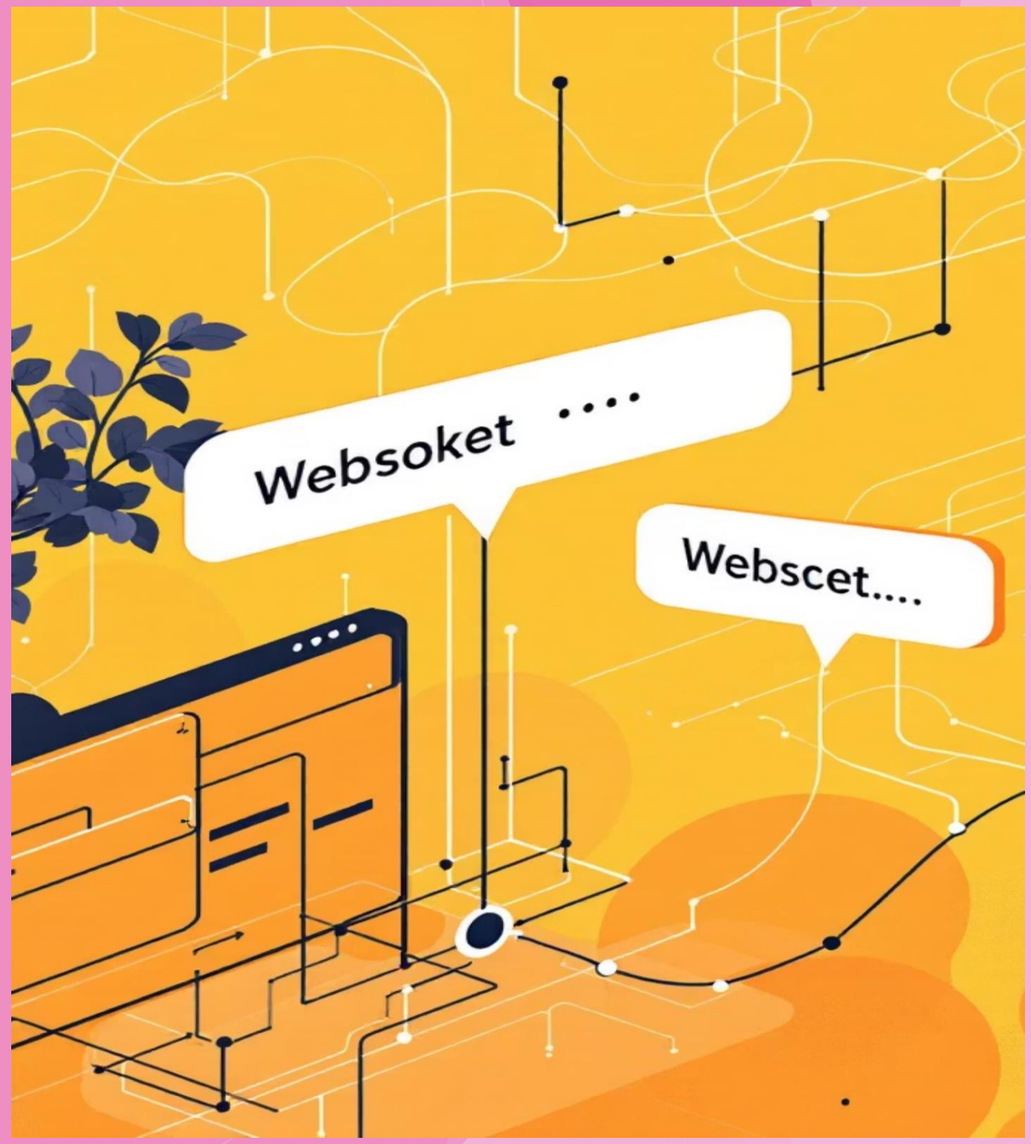
Registration, login and cookie-based session handling.

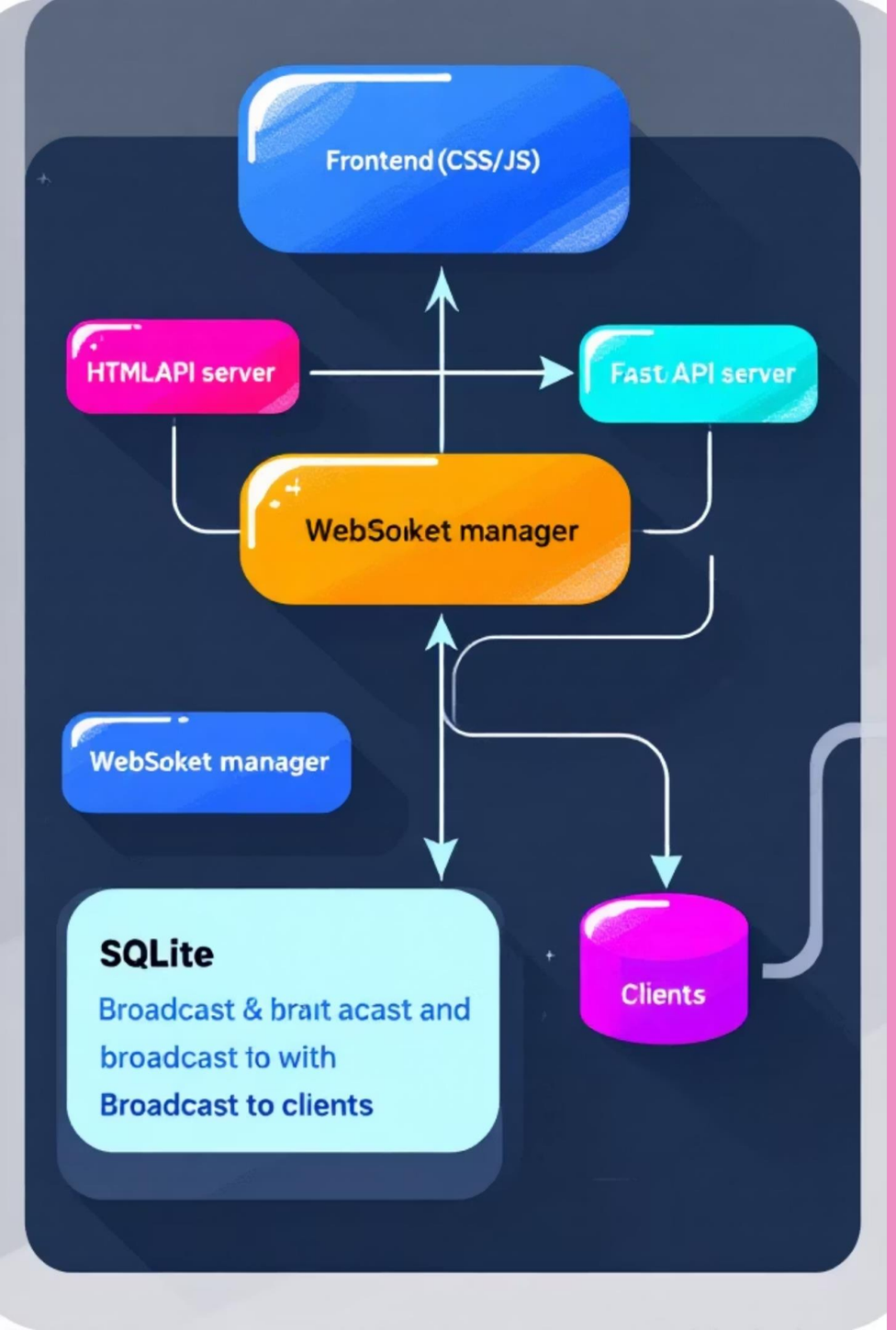
Data Persistence

SQLite stores messages, seen flags and user moderation state.

Moderation & UX

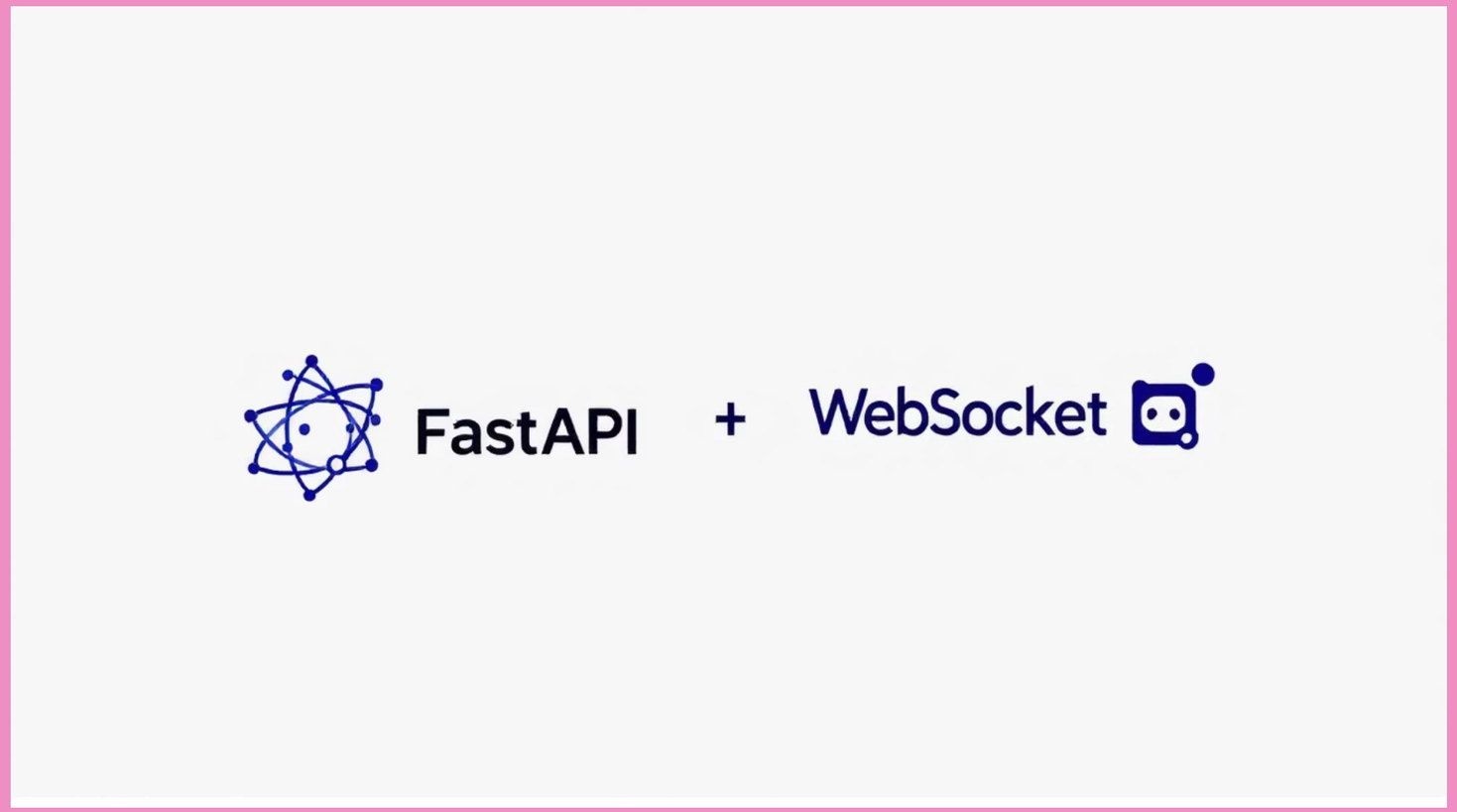
WhatsApp-style seen feature plus toxic-word detection and auto-block.





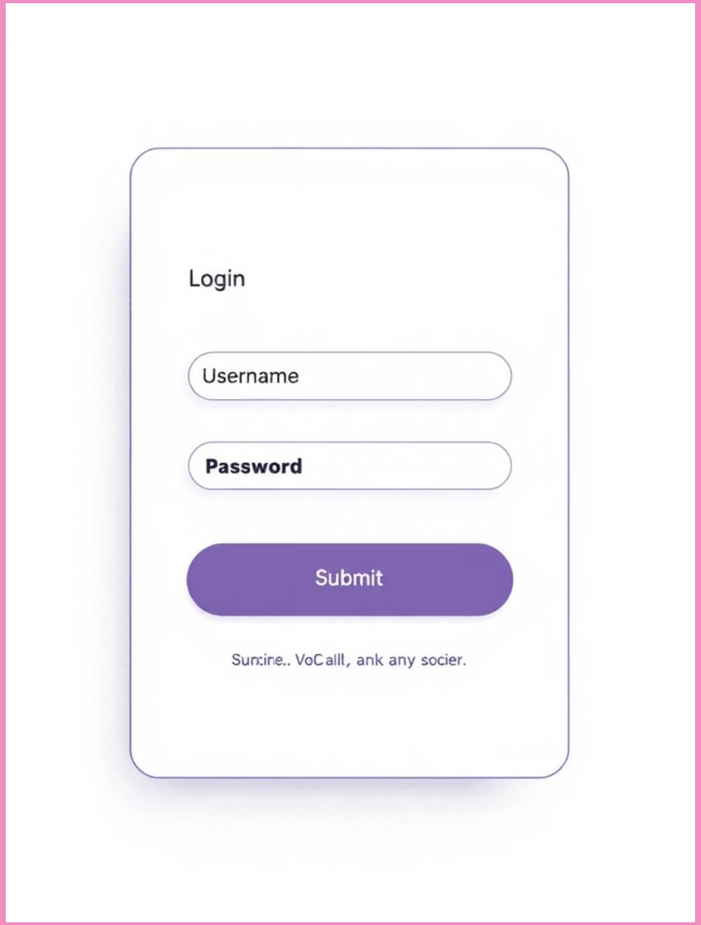
System Architecture

Client (browser) → WebSocket connection → FastAPI backend → SQLite persistence → Broadcast to active users. Frontend uses vanilla HTML, CSS and JavaScript for minimal dependency surface and predictable behaviour.



Technology Stack

- *Backend: FastAPI (Python), native WebSockets for bidirectional events.*
- *Database: SQLite for lightweight, local persistence and simple schema.*
- *Frontend: HTML5, CSS3, JavaScript with WebSocket API for real-time UI updates.*



Authentication Module

Users register and log in; credentials are stored in the users table. Sessions use cookies to maintain authenticated WebSocket connections. The system tracks warnings and blocked state to prevent misbehaviour.

01

1. Register

Validate input and save hashed credentials.

02

2. Login

Verify and issue session cookie; open WebSocket.

03

3. Access Control

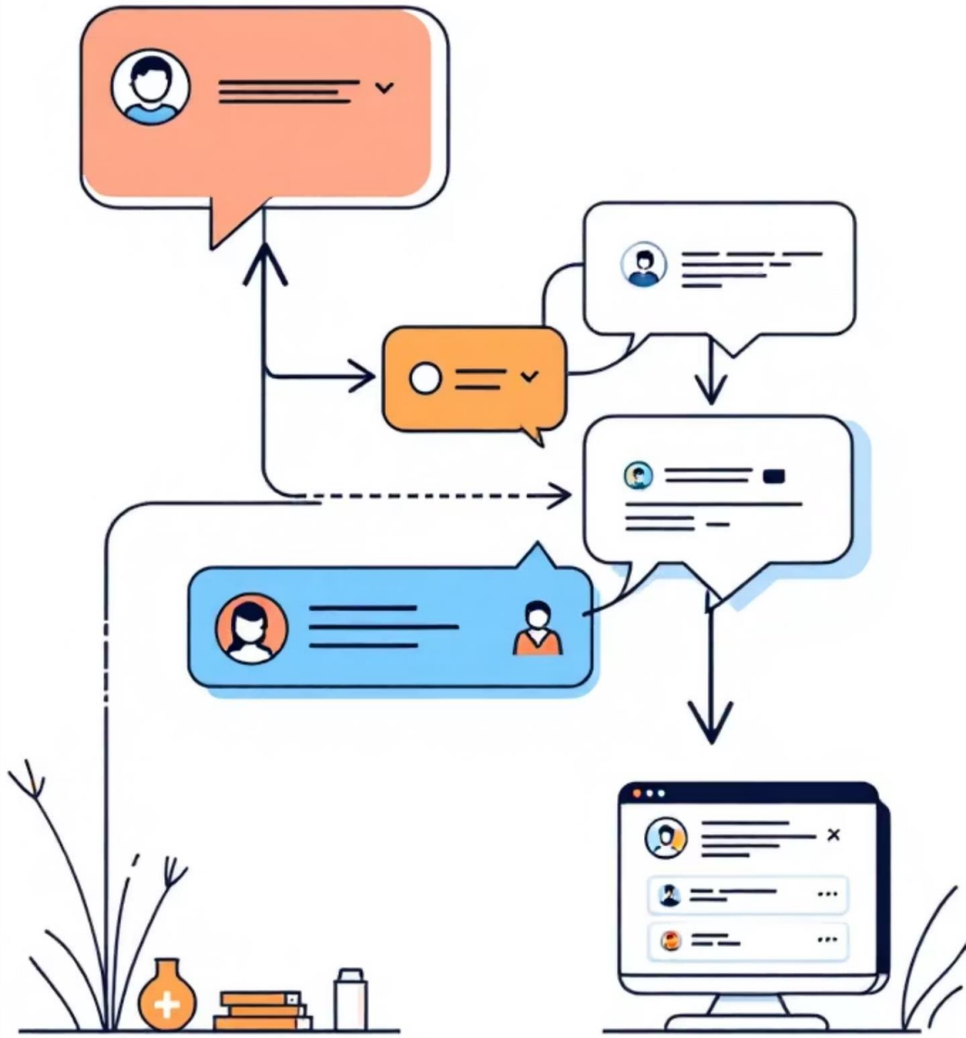
Blocked users are denied send permissions on socket events.

Real-Time Chat Workflow

After login the client opens a WebSocket. Messages are sent as events to the server, written to SQLite, then broadcast to connected clients. The UI updates immediately without page refresh, ensuring smooth, low-latency chat.

- Send Message
- Store Message
- Broadcast
- Update UI

The diagram highlights the core sequence ensuring atomic persistence and delivery to active users.



Seen Status Logic

Visual ticks indicate delivery and read state. Server updates the message 'seen' field when a recipient's client acknowledges the message. UI reflects:

- *Single tick: message sent or delivered but not read.*
- *Double tick: recipient has opened the chat — database updated and sender notified.*

If a recipient is offline or the tab is inactive, the tick remains single until an acknowledgment event is received.

Toxic-Word Moderation

Messages are validated against a predefined stop-word list before broadcast. Enforcement policy is incremental: first and second violations produce warnings; a third violation sets the user's blocked flag in the users table and prevents further sends.

Detection

Server-side check on incoming messages to avoid client bypass.

Enforcement

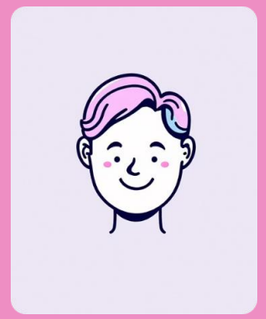
Warnings increment a counter; three strikes → blocked.

UX Feedback

Users receive immediate warnings and an explicit blocked state message.

Data Model & Next Steps

Schema summary:



Users

username (PK), password (hashed), warnings, blocked

Messages

id, sender, message, timestamp, seen

Planned enhancements: group rooms, private chats, avatars, typing indicators and cloud deployment. Thank you.

Thank you